Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

2026
Vladimír Jančár

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

Study Programme: Applied Computer Science
Field of Study: Computer Science
Department: Department of Applied Informatics
Supervisor: Mgr. Ján Pastorek

Bratislava, 2026
Vladimír Jančár

16681876

Comenius University Bratislava
Faculty of Mathematics, Physics and Informatics

THESIS ASSIGNMENT

Name and Surname:	Vladimír Jančár
Study programme:	Applied Computer Science (Single degree study, bachelor I. deg., full time form)
Field of Study:	Computer Science
Type of Thesis:	Bachelor´s thesis
Language of Thesis:	English
Secondary language:	Slovak

Title: Machine learning for generation of graph of given degree and girth.

Annotation: Generating graphs with prescribed structural properties is a key task in combinatorial optimization, network design, and bioinformatics. Traditional algorithmic approaches (e.g., configuration model) often fail to find graphs with high girth and specific degree because the space of possible solutions is vast and the constraints are nontrivial.

Aim: The goal is to investigate potential applications of machine learning to problems in algebraic graph theory. The student will have the opportunity to engage with the state-of-art research in machine learning and algebraic graph theory and contribute to the field by generating new record graphs and training neural networks to predict algebraic properties.

Literature: Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., & Grohe, M. (2019). Weisfeiler and Leman Go Neural: Higher-Order Graph Neural Networks. _Proceedings of the AAAI Conference on Artificial Intelligence_, _33_(01), 4602–4609. [https://doi.org/10/ggfn97] (https://doi.org/10/ggfn97)
Paolino, R., Maskey, S., Welke, P., & Kutyniok, G. (2024). _Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning_.
Wu, L., Cui, P., Pei, J., & Zhao, L. (Eds.). (2022). _Graph Neural Networks: Foundations, Frontiers, and Applications_. Springer Nature Singapore. [https://doi.org/10.1007/978-981-16-6054-2] (https://doi.org/10.1007/978-981-16-6054-2)

Supervisor:	Mgr. Ján Pastorek
Department:	FMFI.KAI - Department of Applied Informatics
Head of department:	doc. RNDr. Tatiana Jajcayová, PhD.
Assigned:	25.02.2025

Approved:

Guarantor of Study Programme

.....
Student

.....
Supervisor

Acknowledgments:

[Acknowledgments to be written near the end of the thesis. Mention the supervisor, people who helped with experiments or computing resources, and any personal thanks that should appear in the submitted version.]

Abstract

[One- to two-sentence framing of the overall goal: machine-learning methods for generating graphs of prescribed degree and girth, i.e. (k, g)-graphs and cage candidates.]

[Paragraph on the preparatory subtasks: which graph-property prediction tasks were studied (degree prediction, minimum-cycle / girth-related prediction), which GNN architectures were compared, and what the headline finding of these preparatory experiments is.]

[Paragraph on the main construction approaches that were tried: direct reinforcement-learning graph editing, voltage graph lifts with non-learned search, and voltage graph lifts with learned policies. Briefly state what each approach contributes.]

[Closing sentence stating the main experimental conclusion and what it implies about the role of GNNs in this setting. To be finalized once the final wording of the main claim is fixed.]

Keywords: [keywords to be finalized, current working set: graph neural networks, algebraic graph theory, cage problem, (k, g)-graphs, voltage graph lifts, heuristic search]

Contents

List of Figures

List of Tables

1 Introduction

This thesis concerns machine-learning methods for generating graphs of prescribed degree and girth. In graph-theoretic language, the central objects are (k, g)-graphs: graphs in which every vertex has degree k and every cycle has length at least g. The degree fixes the local shape of the graph, while the girth says that short loops are forbidden. The classical cage problem asks for such graphs with the smallest possible number of vertices [1].

Practical applications of (k, g)-graphs come from situations where a system should have uniform local connectivity but should avoid short feedback loops. In error-correcting codes, sparse bipartite graphs describe which symbols participate in which parity checks. Decoding algorithms pass messages along this graph. If the graph has many short cycles, the same information can return to a vertex too quickly and the messages become correlated. Larger girth keeps the local neighborhood tree-like for more decoding rounds [2]. Small high-girth regular graphs are therefore useful as compact templates for reliable communication and storage systems.

The same design tension appears in communication networks and supercomputer interconnects. A switch or processor has only a limited number of physical ports, so the degree is constrained by hardware. At the same time, one wants many short routes between machines without local congestion caused by tightly looping connections. Regular high-girth and near-Moore graphs provide blueprints for such networks. The Slim Fly topology, for example, is built from degree-diameter and near-Moore graph ideas and was proposed as a low-cost, low-latency interconnect [3]. In this sense, constructing smaller or better (k, g)-graphs is not only an abstract extremal problem: it enlarges the catalogue of graph topologies available for codes, networks, and other structured systems.

The mathematical task is simple to state but difficult to solve: for fixed k and g, construct a small graph satisfying both constraints. Known approaches use algebraic constructions, voltage graph lifts, exhaustive search, heuristic search, and local modification techniques [4]. This thesis asks whether graph neural networks can help in this setting. The expected role of machine learning is not to replace exact graph-theoretic checks, but to guide a search toward promising choices.

The thesis is organized around the following research questions:

- Which graph neural network architectures can learn the local and cycle-related properties that appear in the definition of a (k, g)-graph?
- Can reinforcement learning construct small (k, g)-graphs by editing edges?
- Does an algebraic representation, in particular voltage graph lifts, give a more useful search space?
- Does learning improve this constrained search, or do non-learned methods remain stronger?

The experiments are organized around a sequence of increasingly constrained formulations. Prediction tasks test whether the models learn local and cycle-related graph information. Direct reinforcement learning then treats construction as an interaction problem, where the model chooses graph edits and receives rewards from the resulting state. Voltage graph lifts provide the most structured formulation: regularity is guaranteed by the base graph, and the search focuses on voltage assignments that avoid short cycles.

2 Background and Definitions

Only the notation needed for the experiments is stated here. More specialized constructions, such as voltage graph lifts, are introduced later together with the methods that use them.

2.1 Graphs

A *graph* is a pair $G = (V, E)$, where V is a finite set of vertices and E is a set of edges. In the main construction setting, graphs are simple and undirected unless stated otherwise.

The *degree* of a vertex $v \in V$, denoted $\deg(v)$, is the number of edges incident with v. The *open neighborhood* of v, denoted $N(v)$, is the set of vertices adjacent to v.

A graph is k-regular if every vertex has degree k.

A *walk* is a sequence of vertices in which consecutive vertices are adjacent.

A *path* is a walk with no repeated vertices.

A *cycle* is a closed path of length at least 3.

A *closed walk* is a walk whose first and last vertices coincide.

A closed walk is *reduced* if it never traverses an edge immediately followed by the same edge in reverse, that is, it never backtracks. Reduced closed walks are the objects used to analyze girth in voltage graph lifts (Chapter 6).

The *girth* of a graph, denoted $\mathrm{girth}(G)$, is the length of its shortest cycle. If a graph has no cycles, its girth is treated as infinite.

A vertex v is *active* if $\deg(v) > 0$. The *active subgraph* of G is the subgraph induced by the active vertices, i.e. G with all isolated vertices removed. This is convenient when a construction algorithm carries a large fixed vertex pool but uses only the vertices it has already drawn into edges.

2.2 (k, g)-Graphs and Cages

A (k, g)-graph is a k-regular graph whose girth is at least g. A (k, g)-cage is a smallest k-regular graph of girth g. Its order is commonly denoted $n(k, g)$. In search algorithms it is convenient to test the condition $\mathrm{girth}(G) \geq g$, because this accepts every valid

candidate for a target lower bound on girth. To compare two constructions, one must also record the number of vertices and the actual girth of the graph found.

The Moore bound gives a general lower bound on the order of a (k, g)-graph. It is useful both as a theoretical benchmark and as a practical target size for construction algorithms.

For $k \geq 2$ and $g \geq 3$, the Moore bound is defined as follows. If g is odd, then

$$M(k, g) = 1 + k \sum_{i=0}^{\frac{g-3}{2}} (k-1)^i.$$

If g is even, then

$$M(k, g) = 2 \sum_{i=0}^{\frac{g-2}{2}} (k-1)^i.$$

A graph meeting this bound is called a Moore graph. Moore graphs are rare. For most parameter pairs, the practical problem is therefore to close the gap between the Moore lower bound and the best published upper bounds.

2.3 Groups

A group $(\Gamma, \cdot)$ is *abelian* if its operation commutes, i.e. $a \cdot b = b \cdot a$ for all $a, b \in \Gamma$. Otherwise it is *non-abelian*. This distinction matters in voltage graph lifts (Chapter 6), where the order of multiplication is part of the construction.

2.4 Graph Neural Networks

Ordinary feed-forward neural networks expect fixed-size vectors. Graphs do not have a fixed number of vertices, and their vertices do not have a canonical ordering. Graph neural networks address this by applying the same local update rule at every vertex. This makes the model independent of the input graph size and equivariant to relabeling of vertices.

Message-passing graph neural networks process graph-structured data by iteratively updating vertex representations through layers [5]. At layer t, each vertex v has a hidden representation h_v^t. The layer aggregates messages from the neighborhood $N(v)$ and then updates the representation:

$$m_v^t = \mathrm{AGG}\big(\{h_u^{t-1} : u \in N(v)\}\big)$$

$$h_v^t = \mathrm{UPD}\big(h_v^{t-1}, m_v^t\big).$$

Several parameters determine what such a model can express. The *number of layers* controls how far information can travel: after one layer, a vertex representation depends on its immediate neighbors, after two layers it can depend on vertices at distance two, and so on. The *hidden dimension* controls how much information

can be stored in each vertex representation. The *aggregation rule* determines which neighborhood statistics are easy to preserve, for example, normalized aggregation can make raw counting harder, while sum-like aggregation keeps count information more directly. Edge features, positional encodings, and graph-level pooling become important when the task depends on edge labels or on the position of a vertex inside the whole graph. This is why local quantities such as degree are easier for standard GNNs than global quantities such as girth.

Grohe's survey on the logic of graph neural networks provides the theoretical framing for this limitation [6]. Standard message-passing GNNs are closely related to color refinement and the one-dimensional Weisfeiler–Leman algorithm. This connection is important for the thesis because it explains why some graph properties are naturally accessible to ordinary GNNs while other global or symmetry-sensitive properties may require stronger architectures, additional features, or a different formulation of the search problem.

The experiments use several GNN architectures:

- *GCN*, a stable baseline based on degree-normalized neighbor aggregation [7].
- *GraphSAGE*, an inductive architecture based on learned aggregation [8].
- *GIN*, a sum-aggregation architecture with expressivity related to the Weisfeiler–Leman test [9].
- *GINE*, an edge-aware variant of GIN used when edge attributes such as voltage labels are part of the input [10].
- *GPS*, a graph transformer architecture combining local message passing with global attention [11]. GPS models often benefit from structural or positional encodings, such as random-walk-based encodings, because attention by itself does not identify a vertex's structural role in the graph.
- *Loopy GNN*, a cycle-aware architecture relevant to minimum-cycle and girth-related tasks [12].

3 Related Work

The work builds on two lines of research: graph-theoretic construction of small regular graphs, and learned heuristics for difficult discrete search problems. The relevant background is not every known cage construction, but the recurring pattern behind successful methods: restrict the search space mathematically, then use computation to explore the remaining choices.

3.1 Classical and Computational Cage Construction

The cage problem has been studied through algebraic constructions, finite geometries, Cayley graphs, voltage graph lifts, exhaustive generation, local search, and excision. The Dynamic Cage Survey provides the broad background and record-holder context [1].

The most relevant lesson from classical construction is that successful methods rarely search over arbitrary graphs. They usually impose structure first and search only inside the remaining degrees of freedom. Algebraic families illustrate this directly. Incidence graphs of finite projective planes and generalized quadrangles yield $(q+1)$-regular graphs of girth 6 and 8 for prime powers q. Generalized hexagons give analogous constructions for girth 12. Cayley graphs on suitable groups produce highly symmetric candidates, and Ramanujan graphs [13] provide explicit algebraic families with optimal spectral properties and large girth. These methods do not search adjacency at all. They encode the constraint into the algebraic object.

Voltage graph lifts continue the same pattern. A small base graph and a finite group define a covering graph whose regularity is inherited from the base. Exoo and Jajcay formalize the relation between voltages on closed walks in the base graph and the girth of the lift [14]. This reduces the search from $O(n^2)$ adjacency choices on a lifted graph to a much smaller voltage assignment on the base.

A second family of methods relies on local search inside the space of k-regular graphs of a fixed order. Hill climbing and tabu search use edge swaps that preserve regularity and minimize a cycle-based cost function. Tabu lists prevent the search from immediately reversing recent moves. Excision methods take a known small cage, remove a carefully chosen subgraph (typically a tree or a small structured neighborhood), and reconnect the remaining deficient vertices to obtain a regular graph one to several vertices below the previous best [15], [16]. Constraint programming and integer programming encodings have also been studied, but the size of the target graphs limits their direct applicability.

Recent computational work combines several of these ideas. Exoo et al. couple voltage-lift generation with advanced tabu search, hill climbing using a distance distribution cost, and tree excision, and report eleven new upper bounds on cage orders in a single pipeline [4]. The takeaway for this thesis is methodological: improvements come from chaining structurally constrained search methods, not from any single algorithm.

These methods provide the natural non-learning baselines. A learned method is not very informative if it only beats unrestricted random edge editing. A stronger comparison asks whether learning improves a search process that already uses graph-theoretic structure, such as a voltage-lift search with exact short-cycle checks.

3.2 Graph Neural Networks and Their Limits

The machine-learning side is closest to work on graph neural networks and learned heuristics for combinatorial search, not only to graph generation. Work such as NeuroSAT showed that neural networks can learn useful signals on symbolic search instances even when they do not replace the exact solver [17]. The analogy here is

that a learned component may guide which states or moves are explored first, while exact algorithms still verify the final graph.

Standard message-passing GNNs are well-suited to permutation-equivariant scoring of graph states, but their expressivity is bounded by the one-dimensional Weisfeiler–Leman test [6], [9]. Garg, Jegelka, and Jaakkola prove that local message-passing GNNs cannot compute several global graph invariants, including girth and diameter, and give the first data-dependent generalization bounds for such models [18]. This is precisely the regime of the cage problem: high girth, vertex-transitive or nearly so, and locally tree-like. On such graphs, L-layer message-passing GNNs collapse to nearly identical vertex embeddings, and standard architectures cannot resolve the global structure required to verify or score a candidate construction.

Two responses to this limit are relevant here. Subgraph-based architectures add explicit cycle or motif counts as structural features, which lifts expressivity above plain 1-WL message passing [19]. Cycle-aware hierarchies such as the Loopy framework [12] extend message passing with cycle bases. Both lines of work are used in this thesis as candidate architectures when the prediction target depends on cycle information.

Degree regularity, on the other hand, can be enforced by construction, and short-cycle violations can often be detected exactly. This motivates learned scoring and move ranking rather than replacing the mathematical verification step.

Reinforcement learning has also been applied directly to extremal graph problems. Wagner combined a cross-entropy method with a small neural policy to construct counterexamples to combinatorial conjectures, on graphs of at most a few tens of vertices with a single scalar objective [20]. The later RLGT framework formalizes the same setting with structural rewards [21]. Where reinforcement learning has succeeded on related problems, the action space already carried structure: Freire et al. apply it to the LDPC components of hypergraph product codes, where the search operates on a constrained construction rather than on arbitrary adjacency [22]. These results frame the construction experiments in this thesis — the unrestricted edge-editing formulation is the hardest case, and the algebraic formulations deliberately reintroduce structure.

4 Probing GNNs on Graph Properties

Degree prediction and minimum-cycle prediction serve as controlled probes for the two structural constraints that define a (k, g)-graph. In both tasks the graph is given and the model assigns a label to each vertex.

Degree prediction is local and exactly verifiable. A vertex degree can be recovered from the immediate neighborhood, so this task tests whether the model, data pipeline, and training loop can learn a simple structural quantity. If an architecture

fails to predict degree reliably, it is not a good candidate for harder construction tasks without further tuning.

Minimum-cycle prediction is a harder structural task. For each vertex, the target is the length of the shortest cycle containing that vertex, with target 0 for vertices that do not lie on any cycle. Unlike degree, this cannot be determined from immediate neighbors alone. It requires information about paths that leave a vertex and later return to it. This makes the task closer to the girth constraint used in constructing (k, g)-graphs.

4.1 Data Generation

Both tasks use dynamically generated random graphs. Instead of training on a fixed set of graphs, new Erdos–Renyi graphs are sampled during training. This reduces the chance that a model simply memorizes a finite training set and gives a cheaper way to expose it to many small graph structures.

Labels are generated by exact graph algorithms. Degree labels are computed as $\deg(v)$ for each vertex. Minimum-cycle labels are computed by searching for the smallest cycle containing each vertex. This keeps the supervision reliable even though the graphs themselves are random.

The input features are intentionally simple. The experiments use either a constant one-dimensional feature or a four-dimensional feature vector consisting of a normalized node index, two random real-valued coordinates, and one additional random scalar. These features should not be interpreted as meaningful graph attributes. They mainly give the neural network a vertex-level input tensor while forcing the model to learn from graph structure.

4.2 Evaluation

The tasks are integer-valued, so a single accuracy number is too coarse: a prediction off by one is qualitatively different from one off by five. Training uses a regression objective with mean squared error, and the results are reported using exact-rounded accuracy together with mean absolute error, root mean squared error, and the fraction of predictions that are off by one.

The evaluation battery contains 75 graphs with 1790 vertices in total. It combines random Erdos–Renyi graphs with structured examples such as complete graphs, complete bipartite graphs, cycles, grids, hypercubes, and known regular graphs such as the Petersen and Heawood graphs. Each trained model is evaluated on the same graphs.

The tables use compact labels. A *small* model has hidden dimension 32 and 4 layers. A *large* model has hidden dimension 128 and 6 layers.

4.3 Degree Prediction Results

The trained variants are denoted *small* (hidden dimension 32, 4 layers) and *large* (hidden dimension 128, 6 layers). Loopy models additionally use a rank-3 cycle basis with weight sharing across layers. Table 1 lists the metrics.

Model	Accuracy	MAE	RMSE
SAGE, small	1.000	0.000	0.000
GIN, small	1.000	0.000	0.000
SAGE, large	1.000	0.000	0.000
GIN, large	0.998	0.002	0.041
GPS-GIN, small	0.998	0.002	0.041
GPS-GIN, large	0.978	0.022	0.148
Loopy, large	0.401	0.925	2.876
Loopy, small	0.366	0.850	1.182
GCN, small	0.336	0.932	1.261
GCN, large	0.307	1.087	1.449

Table 1: Degree prediction metrics by architecture.

GIN and SAGE variants learn degree essentially perfectly at both capacities. GPS-GIN matches them within rounding at the small setting and degrades slightly when made larger. The degree-normalized GCN variants are the weakest, consistent with the expectation that dividing messages by neighborhood size suppresses raw counting information. Loopy GNN, designed to expose cycle information, performs comparably to GCN here: its extra cycle channel does not help on what is essentially a single-hop counting problem.

4.4 Minimum-Cycle Prediction Results

Minimum-cycle prediction is substantially harder. The target asks for the length of the shortest cycle containing each vertex, with value 0 for vertices not contained in any cycle. This requires information beyond the immediate neighborhood and is closer to the girth constraint that appears in constructing (k, g)-graphs. The evaluation uses the same 75-graph battery and the same *small* / *large* capacity conventions as the degree task. Table 2 lists the metrics.

Model	Accuracy	MAE	RMSE	Off by 1
Loopy, large	0.647	1.287	2.834	0.193
Loopy, small	0.393	1.969	3.162	0.182
GIN, small	0.391	1.346	2.125	0.253
SAGE, large	0.266	1.645	2.432	0.319
GPS-GIN, small	0.253	3.382	4.138	0.000
GCN, large	0.188	1.888	2.441	0.255
GCN, small	0.161	2.015	2.533	0.220

Table 2: Minimum-cycle prediction metrics by architecture.

The larger Loopy model is the clear winner, which supports the intuition that cycle-aware architecture is useful for cycle-related tasks. Even so, the task is not solved: its best exact-rounded accuracy is only 0.647, so it still mislabels more than a third of all vertices in the battery. This negative result is useful: it suggests that girth-related information should not be treated as a simple supervised prediction target, and it motivates the later move toward search formulations where exact graph-theoretic checks remain part of the algorithm.

4.5 Lessons From the First Experiments

A model can fail because its aggregation rule is poorly matched to the task, because its capacity is too small, or because it lacks useful structural features. The larger validation run separates these effects more clearly: SAGE solves degree even at small capacity, while the larger Loopy model is the only model that performs substantially better than the rest on minimum-cycle prediction.

The conclusion is that the learning formulation matters. Degree is a local counting problem. Minimum-cycle prediction is a cycle-sensitive structural problem. Constructing a (k, g)-graph is harder still because the model must produce a graph satisfying both constraints, not predict a property of one given.

5 Construction Methods for (k, g)-Graphs

The target problem is to construct small (k, g)-graphs. This is harder than predicting a graph invariant, because the algorithm must actively choose edges while keeping the partial object close to a valid regular graph. The search space grows quickly with the number of vertices, and most arbitrary edge choices lead to invalid or unpromising graphs.

The problem is therefore more naturally viewed as a search problem than as a single prediction problem. A construction algorithm repeatedly holds a partial object, chooses a modification, checks which constraints are still satisfiable, and continues

until it either reaches a valid graph or enters an unpromising region of the search space. Machine learning can enter this process in several different ways: as a learned heuristic for deterministic search, as a policy trained from complete construction attempts, or as a scoring function inside a constrained algebraic search.

5.1 Guided Search as the First Idea

A natural first idea is to use deterministic search and let a GNN guide it. For example, one can build a graph edge by edge and use a learned scoring function to prioritize partial graphs that appear more likely to extend to a valid (k, g)-graph.

The difficulty is not the search loop itself. The difficulty is that ordinary supervised training does not apply: there is no straightforward way to label partial states well enough to train a model to predict whether to expand them. Positive examples can be obtained by taking subgraphs of known valid graphs. Useful negative examples are harder: a partial graph may look bad but still be extendable through further edits. Deciding whether a partial graph can be completed into a valid (k, g)-graph leads back to the same kind of problem. A supervised heuristic can therefore end up training mostly on easy examples far from the decision boundary.

This explains why a purely supervised A-star-style approach was not pursued as the main construction method.

5.2 From Guided Search to Reinforcement Learning

The attractive scenario would be a good heuristic that guides A-star-style search without a pre-built dataset. Reinforcement learning provides exactly this: instead of asking for a dataset that says whether each partial graph is extendable, the model learns from complete construction attempts. The environment supplies rewards based on the observed consequences of actions: whether an edge edit was valid, whether the graph moved closer to regularity, whether short cycles were avoided, and whether the final graph satisfies the target constraints.

In this sense, the reinforcement-learning approach is not separate from search. It is a learned version of heuristic search. A hand-designed search algorithm chooses moves according to a manually specified rule. An RL policy attempts to learn such a rule from repeated interaction with the construction environment.

It is important to keep in mind what this formulation does and does not provide. Individual illegal moves can be ruled out, as the next section describes: an addition that exceeds degree k or closes a cycle shorter than g is simply never offered to the agent, so every intermediate graph stays legal. What no such rule provides is completion. Unlike the algebraic constructions of the next chapter, where regularity is automatic, nothing here guarantees that a sequence of legal edits ever reaches a graph that is k-regular on enough vertices. The agent must learn to assemble one edge by edge rather than stall in a legal but unfinishable state. The direct-RL formulation

is therefore useful partly as a diagnostic: it tests how far a learned search can go when local legality is enforced but global completion is left to the policy.

5.3 Direct Reinforcement Learning

In the direct reinforcement-learning formulation, construction is represented as a sequential decision problem and trained with Proximal Policy Optimization [23]. The state is a partially constructed graph. Each action selects an unordered pair of vertices. If the edge is absent, the action attempts to add it. If the edge is present, the action attempts to remove it.

This direct graph-editing formulation has an advantage: it does not require a precomputed dataset of labeled partial graphs. The model learns from interaction. However, it also makes the action space and the reward design essential, and these are addressed in the next subsections.

Several invalid or unhelpful actions are removed from the action space before the policy sees them, so the agent cannot select them at all:

- An edge cannot be added if either endpoint already has degree k.
- An edge cannot be added if it would create a cycle shorter than g.
- An edge cannot be removed if doing so would disconnect the active subgraph.
- An edge cannot be removed if it would leave too few active vertices to reach the Moore-bound target.

5.4 Curriculum Learning

The motivation for the curriculum is direct: at uniform difficulty an untrained agent almost never produces a valid graph, so the learning signal collapses and nothing is learned. Starting from parameter pairs with a small Moore bound raises the per-episode success rate enough that the agent has something to learn from, and the difficulty can then be increased once that signal is reliable.

The environment orders parameter pairs by their Moore bound and unlocks the next pair only after the agent solves at least half of its last eight episodes on the current pair. Training starts from $(3, 5)$.

Curriculum learning is what makes the direct RL formulation learn at all. Without it, the agent never produces a valid $(3, 5)$-graph. With it, the agent reaches and solves the first stages, the $(3, 5)$-graph and the $(3, 6)$-graph. Reaching even these stages also depended on the reward design described next.

5.5 Reward Shaping

The environment uses a shaped reward. Earlier reward designs concentrated almost all signal at the end of an episode, and successful episodes were too rare for that signal to guide learning reliably. A per-step progress term was added so that the agent receives a positive signal when an action moves the partial graph closer to

a valid construction and a negative one when it moves away. A valid final graph receives a large terminal reward, but the per-step term carries most of the learning signal.

The progress term is implemented as a potential-based shaping signal [24]. Let s denote a state (a partial graph) and let $\Phi(s)$ denote its *potential*: a scalar that grows as the active subgraph approaches a k-regular graph on the Moore-bound number of vertices, rewarding both each active vertex approaching degree k and the total edge count approaching the corresponding $kM/2$ edges. The per-step shaping reward is

$$\Phi(s') - \Phi(s),$$

where s' is the state reached after taking the action in state s. The value is positive when the partial graph moves closer to the target and negative when it moves away.

5.6 Limits of the Direct Formulation

The direct graph-editing formulation is useful as a baseline, but it has structural weaknesses. Regularity is not guaranteed. It must be learned or enforced through action pruning and rewards. The action space also grows quadratically with the number of available vertices: on a graph with n vertices, there are $\binom{n}{2} = \frac{n(n-1)}{2}$ possible unordered pairs, and each pair can become an add-or-remove decision depending on the graph state.

These limits motivate the voltage-lift formulation of the next chapter, which builds regularity into the representation rather than leaving it to the agent.

6 Voltage Graph Lifts

Voltage graph lifts are an alternative construction formulation in which k-regularity becomes structural: every lift of a k-regular base graph is itself k-regular. The control problem then reduces to choosing group labels on the edges of a small base graph instead of choosing every edge of the large lifted graph, and the learned components introduced later in this chapter operate inside this reduced space.

6.1 Parameters of the Construction

A voltage lift is determined by three choices, and only three. Everything else is forced once they are fixed.

The first choice is a finite group Γ. Its role is to set how many copies of the base graph the lift contains: the lift consists of $|\Gamma|$ copies of B, so a larger group produces a larger graph.

The second choice is an orientation of each base edge. This is only a bookkeeping convention for the direction in which a voltage is read. Reversing an arc and replacing its voltage a by the inverse a^{-1} produces exactly the same lift.

The third choice is the voltage assignment α, which labels each oriented edge with an element of Γ. This is the only choice that affects girth. It is precisely the voltage assignment that the search and learning procedures set out to optimize.

What is not free follows from these three. The number of vertices is $|V(B)| \cdot |\Gamma|$, the degree of every lift vertex equals the degree of the base vertex it copies, and the adjacencies of the lift are fixed by the rule in the next subsection.

6.2 Building the Lift

The base graph B is small and may contain loops and parallel edges. For undirected graphs each edge is represented by two opposite arcs. If one direction carries voltage a, the reverse carries a^{-1}.

The lift has vertex set $V(B) \times \Gamma$. Writing (v, h) for the copy of base vertex v on layer $h \in \Gamma$, an arc from u to v with voltage a contributes, for every group element h, the edge

$$(u, h) \quad \text{to} \quad (v, ha).$$

This thesis uses right multiplication by the voltage element. For abelian groups the order of multiplication is invisible, but it matters for non-abelian groups.

As an example, take $B = K_4$ and $\Gamma = Z_3$, so the lift has $4 \cdot 3 = 12$ vertices. Figure 1 shows an assignment in which a spanning star carries voltage 0 and the remaining triangle carries voltage 1. An edge with voltage 1 from u to v produces (u, h)–$(v, h + 1)$ for $h \in \{0, 1, 2\}$, so each base edge becomes three lift edges and the three layers are stitched together cyclically. The resulting graph is 3-regular and connected.

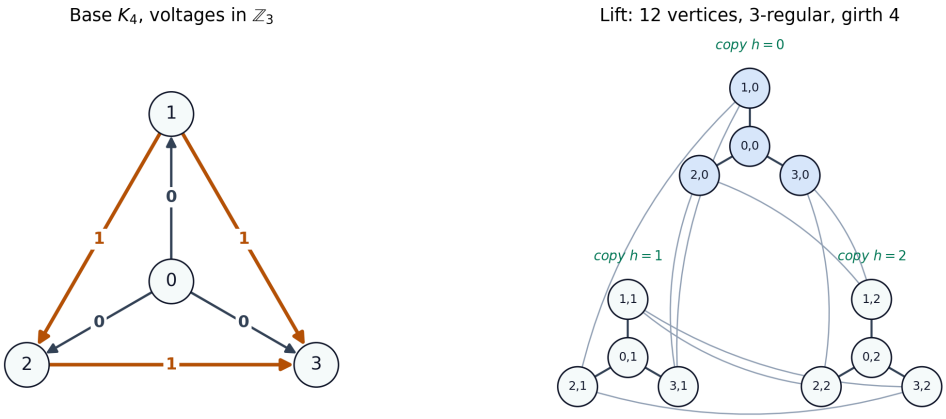

Figure 1: A voltage lift of K_4 over Z_3.

6.3 Regularity Is Structural

The reason the construction guarantees k-regularity is direct. Fix a base vertex u of degree k and a layer h. Each arc leaving u contributes exactly one lift edge at (u, h):

the arc to base neighbor v with voltage a produces the single edge (u, h)–(v, ha). The k arcs at u therefore yield exactly k edges at (u, h), regardless of which voltages were chosen, so every lift vertex has degree k.

This is the property that makes the formulation attractive for (k, g)-graphs. In direct edge-by-edge generation the agent must continually maintain the degree of every vertex. Here the degree constraint holds automatically, and only the girth constraint remains to be controlled through the voltage assignment.

6.4 Computing Girth on the Base Graph

The second advantage is that girth can be read off the base graph without building the lift. Consider a reduced closed walk W in the base graph. Multiplying the voltages along its arcs (using a^{-1} for an arc traversed backward) gives the *net voltage* of W.

Following the corresponding lift edges around W from layer h returns to the starting base vertex, but on layer hs, where s is the net voltage. Two cases arise:

- If s is the identity, the walk closes after one trip and lifts to a cycle of the same length as W.
- If s is not the identity, the walk must be repeated until the accumulated layer shift returns to the identity. The number of repetitions is the order of s, so W lifts to a cycle of length $|W| \cdot \mathrm{ord}(s)$.

The girth of the lift is the length of the shortest cycle obtained this way:

$$\mathrm{girth}(\mathrm{lift}) = \min_{W} |W| \cdot \mathrm{ord}(s),$$

minimized over reduced closed walks W of the base graph, where s is the net voltage of W — equivalently, the shortest base walk whose net voltage is the identity.

Figure 2 illustrates both cases on the running example. Every triangle of K_4 has net voltage 1, which has order 3 in Z_3. A triangle therefore cannot close after one trip and instead wraps three times into a single 9-cycle that visits all three layers. The 4-cycle 0–2–3–1, by contrast, has net voltage 0 and survives as an ordinary 4-cycle. It is the shortest surviving cycle, so the lift has girth 4, raising the girth of 3 in K_4. No voltage assignment on K_4 over Z_3 reaches girth 5. Higher girth requires a larger group or a different base graph, which is why the search later ranges over many base–group pairs.

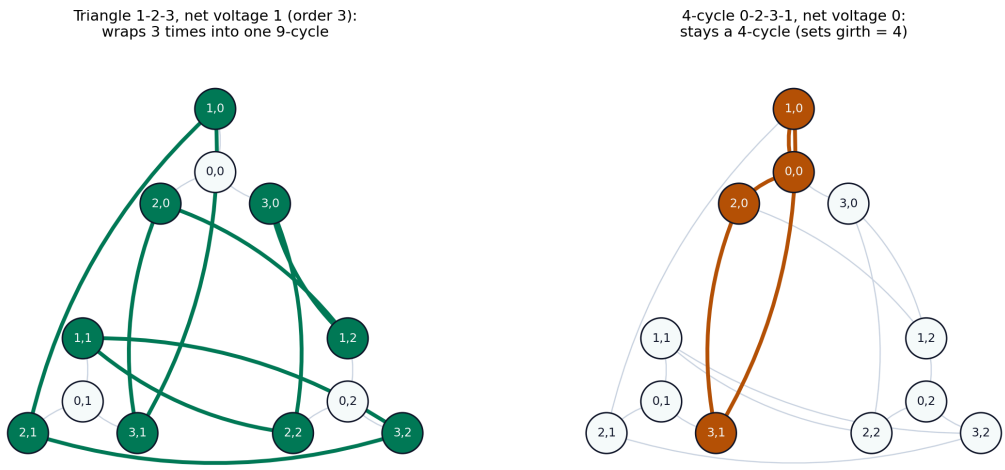

Figure 2: How base cycles determine the girth of the lift.

This analysis gives a cheap exact cost for search: enumerate the reduced closed walks shorter than the target girth g and count those with identity net voltage. The count is zero exactly when the lift has girth at least g, and it is computed on the small base graph rather than on the full lift. This exact cost is used by the search and reinforcement-learning procedures below, and it is the main reason the learned component can sit inside a constrained search instead of generating an entire graph from scratch.

6.5 Reducing the Search Space

The search becomes cheaper when fewer edges need a voltage: exhaustive or guided search over m undirected base edges scales with $|\Gamma|^m$, so any voltage that can be fixed in advance, without ruling out a single lift, shrinks the problem. Several base voltages can indeed be fixed for free, and this subsection explains which ones and why.

The reason such fixing is harmless is that the labels attached to the copies of a base vertex are arbitrary. At a vertex v, the $|\Gamma|$ copies may be renamed among themselves. This is a pure relabeling of lift vertices, so the lifted graph is unchanged up to isomorphism. Renaming the copies of v by a group element $\tau(v)$ shifts the voltage of every arc touching v: it adds τ at the arc's tail and subtracts it at the arc's head, so an arc from u to v acquires the new voltage

$$\alpha(u \to v) + \tau(u) - \tau(v).$$

Choosing one such element $\tau(v)$ at every vertex, and letting it act on the voltages in this way, is called a *gauge transformation*.

A gauge transformation cannot change the girth, because it leaves the net voltage of every closed walk untouched. Adding up the corrections along a closed walk, each vertex it passes through is entered along one arc — contributing $-\tau$ — and left along

the next — contributing $+\tau$. The two cancel. Over the whole walk the corrections sum to $\tau(\text{start}) - \tau(\text{end}) = 0$. Since girth is determined entirely by the net voltages of closed walks, the lift is the same graph before and after.

This freedom can now be spent to drive voltages to zero. Forcing a single arc to zero means choosing the potentials so that

$$\alpha(u \to v) + \tau(u) - \tau(v) = 0.$$

The catch is that not every edge can be zeroed at once, and the reason is exactly the cancellation above: around any cycle the τ corrections sum to zero, so the net voltage of a cycle is fixed — a gauge transformation cannot touch it. If a cycle carries a nonzero net voltage, its edges therefore cannot all be made zero. Cycles are the obstruction, and only a set of edges that contains no cycle can be zeroed simultaneously.

This is what singles out a spanning tree. A spanning tree is the largest set of edges of a connected graph that contains no cycle, and it has $|V(B)| - 1$ edges. Its voltages can always be cleared: root the tree, set $\tau = 0$ at the root, and work outward. Each tree edge joins an already-fixed vertex u to a new vertex v, and the choice $\tau(v) = \alpha(u \to v) + \tau(u)$ makes that edge's voltage zero. Because the tree has no cycle, every vertex is reached exactly once, along a unique path from the root, so these choices never conflict. Adding any further edge would close a cycle, whose net voltage is beyond reach, so a spanning tree is as far as the zeroing can go.

What remains are the $m - |V(B)| + 1$ non-tree edges — the only edges that carry genuine freedom. The effect is substantial. For the example, K_4 has six edges and a spanning tree of three, so three voltages are fixed to zero and only three remain to be chosen: the triangle on vertices $1, 2, 3$ in the assignment above. The search assigns three voltages instead of six.

6.6 Search Methods over Voltage Assignments

Several search strategies share the voltage-lift formulation. They differ in how they choose voltage assignments, but they all keep exact verification at the end: a generated graph is accepted only after connectedness, regularity, and girth are checked.

For small groups and few base edges, exhaustive search enumerates every voltage assignment. This is only practical when $|\Gamma|^m$ is small, where m is the number of undirected base edges. Random search samples assignments and verifies them. It is simple, but it is an important baseline because some small cases are easy enough that random search succeeds.

The tabu search follows recent computational work on small regular graphs [4]. It fixes tree-edge voltages to the identity, so only non-tree edges are searched. It then changes one voltage at a time and minimizes the number of short identity-voltage walks. A tabu list prevents the algorithm from immediately undoing recent moves.

The GNN-guided beam search assigns voltages edge by edge. A trained girth-predictor network scores partial assignments, and the search keeps only the highest-scoring candidates. *[Architecture and evaluation of this predictor to be detailed in Chapter 9.]* A meta-search procedure additionally enumerates choices of base graph and finite group. For cubic graphs, the candidates include the dumbbell base, a four-node cubic base, the prism base, and a Petersen-like base. Candidate groups include cyclic groups, dihedral groups, direct products, and semidirect products.

The reinforcement-learning environment treats one voltage assignment as one episode. The state is the base graph with partial voltage labels, the action is a group element, and the episode length is the number of base edges. This is much shorter than the direct edge-editing environment. The policy model uses either a GINE-only encoder or a GPS encoder with GINE as the local message-passing component, followed by global pooling, an actor head over group elements, and a critic head for PPO.

7 Refinement and Excision

This chapter describes two methods that act on an already-constructed k-regular graph rather than building one from scratch. Edge-swap refinement edits a graph to remove short cycles while preserving its degree, and excision removes vertices from an existing graph to obtain a smaller one.

7.1 Edge-Swap Refinement

A voltage lift always returns a k-regular graph, but its girth may fall short of the target g, since a few short cycles can survive the construction. Such a candidate is close to a solution, and discarding it wastes the work already spent. The refinement step instead edits a small number of edges to remove the surviving short cycles while keeping the graph k-regular.

This reverses the difficulty seen during construction. There, regularity was automatic and girth was the hard constraint; here the graph already has the right degree, so the edit must improve girth without breaking regularity. That calls for a move which never changes any vertex degree.

The basic move is the 2-swap. Take two edges with no shared endpoint, say A–B and C–D on four distinct vertices, delete both, and reconnect the same four vertices the other way, for example as A–D and B–C (Figure 3). Each of the four vertices loses one edge and gains one, so every degree is preserved and a k-regular graph stays k-regular. Four vertices joined by two disjoint edges admit three perfect matchings, so from the original pairing there are two alternative 2-swaps available.

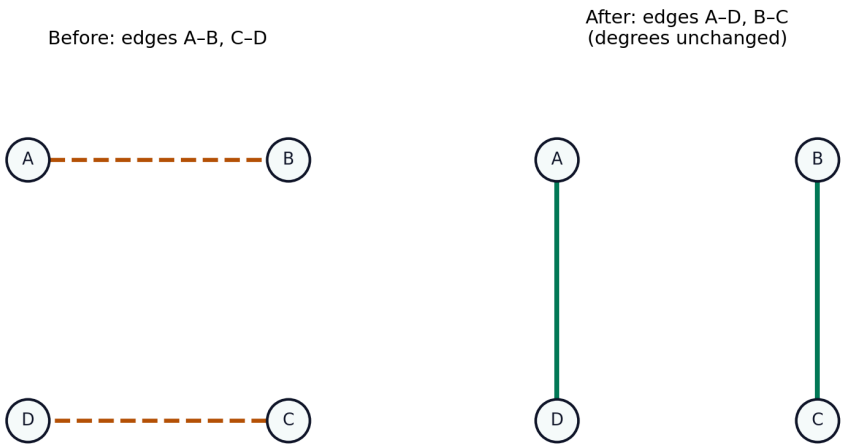

Figure 3: A 2-swap. The two disjoint edges A–B and C–D (left, dashed) are replaced by A–D and B–C (right). Every vertex keeps its degree, so regularity is preserved.

A 2-swap can destroy a short cycle. To remove a triangle A–B–C, swapping its edge A–B against a disjoint edge D–E deletes A–B and breaks the triangle, again without changing any degree (Figure 4). The difficulty is that deleting one edge and adding others creates new adjacencies, which may form new short cycles elsewhere. A single swap can therefore help in one place and hurt in another, so refinement is a search over many swaps rather than a one-shot edit.

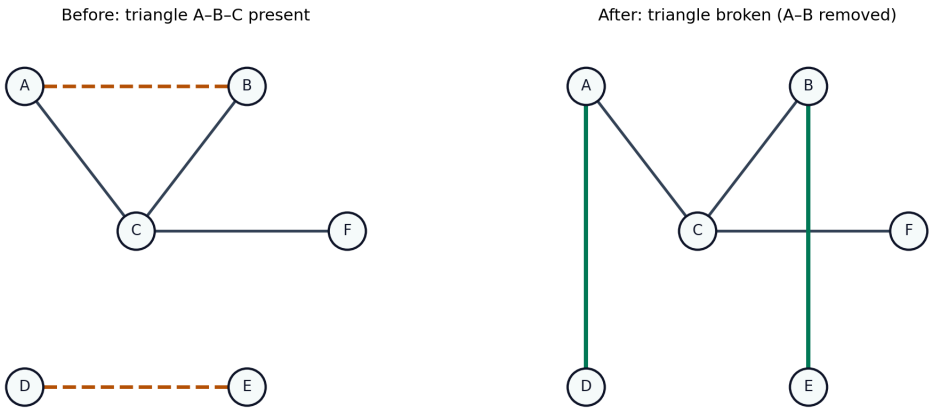

Figure 4: A 2-swap removing a short cycle. The triangle A–B–C and a disjoint edge D–E (left) are swapped to A–D and B–E (right): the edge A–B is gone, so the triangle is broken, and all degrees are intact.

Girth itself is a poor signal to steer this search. It reports only the length of the single shortest cycle, so removing one of several equally short cycles leaves it unchanged, and the search sees a flat plateau with no feedback on partial progress. Instead the search minimizes a weighted count of all short cycles,

$$f(G) = \sum_{c=3}^{g-1} w_c \cdot N_c(G), \quad w_c = 2^{g-c},$$

where $N_c(G)$ is the number of cycles of length c. Shorter cycles receive exponentially larger weight, so a triangle counts far more than a near-target cycle and is removed first. The cost moves on almost every swap, giving the search a usable gradient, and it reaches 0 exactly when no cycle shorter than g remains, that is, when the girth target is met.

The search applies swaps that lower f, keeping a short tabu list of recent moves that may not be reversed immediately. This stops the search oscillating between two states and, at a local minimum, lets it accept a temporarily worse move to escape, since the reverse is forbidden for a few steps and cannot pull it straight back. When the 2-swap neighborhood is exhausted and the cost is still positive, the search escalates to 3-swaps, in which three pairwise-disjoint edges are rewired at once into a new degree-preserving matching (Figure 5). A 3-swap reaches configurations that no single cost-reducing 2-swap can, crossing barriers that trap the smaller move. Because there are $O(|E|^3)$ candidate 3-swaps against $O(|E|^2)$ 2-swaps, they are used only as an occasional escalation when the search stagnates, not as the routine move.

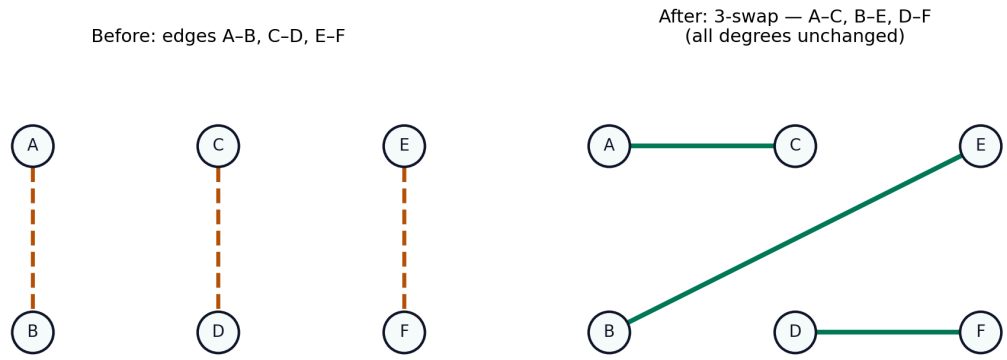

Figure 5: A 3-swap. Three disjoint edges A–B, C–D, E–F (left) are rewired to A–C, B–E, D–F (right). All six vertices keep their degree.

Scoring candidate swaps is the cost bottleneck. There are $O(|E|^2)$ possible 2-swaps each iteration, and evaluating one exactly means recounting the short cycles it changes. A trained evaluator network replaces this with a cheaper estimate: a single pass of a graph neural network produces an embedding for every vertex, and the predicted change in cost for a swap is then read from the embeddings of its few endpoints through a small head. The expensive graph reasoning is computed once per iteration and shared across all candidate swaps, which are scored together in one batched pass. The network uses cycle-count features and random-walk positional

encodings, which break the symmetry that otherwise makes vertices indistinguishable on highly regular graphs.

Because the estimate is approximate, it is used only to rank candidates: the search verifies the most promising swaps exactly before committing to one. The approximation therefore affects only how quickly and how well the search proceeds, never the validity of the result, since every accepted graph is checked exactly against the girth target. This is the same division of labor as in the construction stage, where a learned component guides the search and exact verification decides.

This refinement is the most recent component of the pipeline, and its full evaluation is still in progress. An initial training run on 100{, }000 generated swap labels reduced the mean squared error of the move evaluator by roughly 30% relative to its initialization. Held-out evaluation, a comparison against the unguided search, and feature ablations are reported in Chapter 9.

7.2 Excision

[Placeholder — to be written together: the excision method. A girth-safe BFS tree of depth $\left\lfloor \frac{g-1}{2} \right\rfloor$ *is removed from a known* (k, g)-graph*, leaving boundary vertices of deficient degree; repairing them with girth-safe edges yields a smaller* (k, g)-graph*. Include worked examples in the style of the earlier chapters.]*

8 Forge

[Placeholder — to be written: the Forge pipeline, which chains the methods of the previous chapters. A voltage search (currently the reinforcement-learning variant) produces candidates and is deliberately allowed to pass near-perfect graphs whose girth is just below g; edge-swap refinement (Chapter 7) then closes the remaining gap; and excision (Chapter 7) reduces the vertex count. Describe the stage hand-offs, the design rationale for the near-perfect hand-off, and why this ordering. Reference Chapters 6 and 7 rather than re-explaining the components.]

9 Results

[This chapter evaluates all (k, g)-graph construction approaches together — direct reinforcement learning, voltage-lift search and its learned variants, and edge-swap refinement — on a common set of targets. To be written once the comparative validation runs are complete: report which approaches solve which targets, the order of the graphs found relative to the Moore bound and known cage orders, and the search cost of each method.]

10 Implementation

This chapter describes how the methods presented in the earlier chapters are realized in practice, covering the organization of the software, training procedures, and configuration.

10.1 Code Organization

[Placeholder — to be written: how the project is structured into modules, the separation between graph construction, search, and learning components, and the interfaces between them.]

10.2 Training and Reproducibility

[Placeholder — to be written: how models are trained, which hyperparameters are exposed, how seeds and checkpoints are managed, and how experiments can be reproduced.]

10.3 Configuration

[Placeholder — to be written: how the system is configured for different (k, g) targets, how base graphs and groups are selected, and how search parameters are set.]

11 Future Work

[To be written near the end. Candidate directions to cover: stronger voltage-lift search (pruning, canonicalization) before learning is applied, better training signals and hard-negative mining for the learned scorers, rewards that favor minimality (smaller lift orders / cage-finding) rather than just feasibility, training the excision-repair policy properly and adding excision as a full construction method, and learning as move-ranking inside existing search rather than end-to-end generation.]

12 Conclusion

[To be written last, once Chapter 9 is settled. Should summarize: the question (can GNNs help generate (k, g)-graphs), the preparatory finding (local vs. cycle-related properties), the construction methods tried (direct RL, voltage lifts and learned variants, edge-swap refinement), and the overall verdict on where learning helps relative to structured non-learned search. Keep the final claim aligned with what Chapter 9 actually shows.]

Bibliography

[1] G. Exoo and R. Jajcay, "Dynamic Cage Survey," *The Electronic Journal of Combinatorics*, 2013.

[2] R. M. Tanner, "A Recursive Approach to Low Complexity Codes," *IEEE Transactions on Information Theory*, vol. 27, no. 5, pp. 533–547, 1981, doi: 10.1109/TIT.1981.1056404.

[3] M. Besta and T. Hoefler, "Slim Fly: A Cost Effective Low-Diameter Network Topology," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2014.

[4] G. Exoo, J. Goedgebeur, J. Jooken, L. Stubbe, and T. Van den Eede, "New small regular graphs of given girth: the cage problem and beyond." 2025.

[5] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, 2017.

[6] M. Grohe, "The Logic of Graph Neural Networks." 2022.

[7] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *International Conference on Learning Representations*, 2017.

[8] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in Neural Information Processing Systems*, 2017.

[9] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?," in *International Conference on Learning Representations*, 2019.

[10] W. Hu *et al.*, "Strategies for Pre-training Graph Neural Networks," in *International Conference on Learning Representations*, 2020.

[11] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a General, Powerful, Scalable Graph Transformer," in *Advances in Neural Information Processing Systems*, 2022.

[12] R. Paolino, C. Vignac, and A. Loukas, "Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning," in *International Conference on Learning Representations*, 2024.

[13] A. Lubotzky, R. Phillips, and P. Sarnak, "Ramanujan graphs," *Combinatorica*, vol. 8, no. 3, pp. 261–277, 1988, doi: 10.1007/BF02126799.

[14] G. Exoo and R. Jajcay, "On the girth of voltage graph lifts," *European Journal of Combinatorics*, vol. 32, no. 4, pp. 554–562, 2011, doi: 10.1016/j.ejc.2010.11.011.

[15] G. Araujo-Pardo, C. Balbuena, and J. C. Valenzuela, "Constructions of bi-regular cages," *Discrete Mathematics*, vol. 309, no. 14, pp. 4844–4853, 2009, doi: 10.1016/j.disc.2008.04.011.

[16] G. Exoo, "On decreasing the orders of (k, g)-graphs", *Journal of Combinatorial Optimization*, 2023, doi: 10.1007/s10878-023-01092-9.

[17] D. Selsam, M. Lamm, B. Bünz, P. Liang, L. de Moura, and D. L. Dill, "Learning a SAT Solver from Single-Bit Supervision," in *International Conference on Learning Representations*, 2019.

[18] V. K. Garg, S. Jegelka, and T. Jaakkola, "Generalization and Representational Limits of Graph Neural Networks," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[19] G. Bouritsas, F. Frasca, S. Zafeiriou, and M. M. Bronstein, "Improving Graph Neural Network Expressivity via Subgraph Isomorphism Counting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.

[20] A. Z. Wagner, "Constructions in combinatorics via neural networks." 2021.

[21] I. Damnjanović, U. Milivojević, I. Đorđević, and D. Stevanović, "RLGT: A reinforcement learning framework for extremal graph theory." 2026.

[22] B. C. A. Freire, N. Delfosse, and A. Leverrier, "Optimizing hypergraph product codes with random walks, simulated annealing and reinforcement learning." 2025.

[23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[24] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999.